INDEX

Name: DEEPASHRI MURALI Roll No: 240701100

[3rd yr]

Std: CSE Subject: Computer Networks School/College:

Sl. No.	Date	Title	Page No	Remarks
1.	20/7/26	Study of various Network commands used in LINUX and windows		Dawn 22/7/26
2.	27/7/26	Study of different types of Network cables		9 Dawn 4/8/26 9
3	3/8/26	CISCO PACKET TRAILER	9	18/8/26
4	10/8/26	Setup and configure a LAN using switch and ethernet cables	9	18/8
5	18/8/26	WIRESHARK	9	7/9
6	7/9/26	Error Detection & correction using hamming Code	10	22/9
7	22/9/26	NMAP to discover live hosts using NMAP scans on the Tryhackme Platform	10	28/9
8	28/9/26	End to End communication at transport layer	10	29

Sl. No.	Date	Title	Page No	Remarks
9	28/9/26	Sliding Window Protocol	9	
10	29/9/26	Implementation of Subnetting in Cisco Packet Tracer Simulator		
11	5/10/26	Internetworking with Routers with Cisco packet tracer simulator		
12	13/10/26	Routing at Network layer		
13	13/10/26	Implement your own Ping Program		
14	13/10/26	Packet sniffing using Raw Sockets		

20/7/26

PRACTICAL - 1

STUDY OF VARIOUS NETWORK COMMANDS USED IN LINUX AND WINDOWS

Basic networking commands

① arp-a :- ARP is short form of address resolution protocol
It will show IP address of your computer along with IP address & MAC address of router

② hostname :- This is the simplest of all TCP/IP commands
It simply displays the name of your computer

③ ipconfig /all :- This command displays detailed configuration information about your TCP/IP connection

④ nbstat -a :- this command helps in solving problems with NetBIOS name resolution

⑤ netstat :- [Network statistics] It displays a variety of statistics about computers active TCP/IP connecti

It is a command line tool for monitoring network connections both incoming and outgoing as well as viewing routing tables, interface statistics etc

Eg:- netstat -r

⑥ nslookup :- Tool used for performing DNS lookups in Linux. It is used to display DNS details

Nslookup can operate in two modes : interactive & non-interactive

Eg:- nslookup www.google.com

ot on OnePlus

⑦ Pathping : Pathping is unique to Window's and is basically a combination of the Ping and Tracert commands.

Pathing traces the route to the destination address then launches a 25 sec test of each router along the way, gathering statistics on the rate of data loss along each hop

⑧ Ping :- [Packet INternet Groper] - best way to test connectivity b/w two nodes

ICMP [Internet control Message Protocol] to communicate to other devices

Ping hostname (ping localhost)

ping ip address (ping 4.2.2.2)

ping fully qualified domain name
(ping www.facebook.com)

⑨ Route :- This command is used to show the IP routing table. It is primarily used to setup static routes to specific host or network via interface

Some important linux networking commands

① ip

ip <options> <objects> <command>

② [root@server ~] # ip address show

Show IP address assigned to an interface

③ [root@server ~] # ip address add 192.168.1.254/24 dev enp0s3

assign an IP to an interface

④ [root@server ~] # ip address del 192.168.1.254/24 dev enp0s3

To delete an IP on an address

⑤ [root@server ~] # ip link set eth0 up

Alter the status of interface by bringing the interface eth0

⑥ [root@server ~] # ip link set eth0 down

Alter the status of the interface by bringing the interface eth0

⑦ eth0: [root@server ~] # ip link set eth0 promisc on

Alter the status of interface by enabling Promiscuous mode

⑧ eth0: [root@server ~] # ip ~~link set~~ route add default via 192.168.1.254 dev eth0

⑨ Add a route to 192.168.1.0/24 via gateway at 192.168.1.254

[root@server ~] # ip route add 192.168.1.0/24 via 192.168.1.254

⑩ eth0: [root@server ~] # ip route add 192.168.1.0/24 dev eth0

Add route that can be reached on device

① [root@server ~] # ip route delete 192.168.1.0/24 via 192.168.
Delete the route 192.168.1.0/24 via gateway 192.168.1.2

② [root@server ~]# ip route get 10.10.1.4
Display the route taken for IP 10.10.1.4

② ifconfig :-
Tool for configuring and troubleshooting networks

③ mtr :-
mtr <options> hostname/ IP

ⓐ [root@server ~]# mtr google.com
Basic mtr command shows you the statistics, including each hop with time & loss %

ⓑ [root@server ~] # mtr -g google.com
show numeric IP address

ⓒ [root@server ~] # mtr -b google.com
show numeric IP addresses & hostnames

ⓓ [root@server ~] # mtr -c 10 google.com
Set no of pings that you want

④ tcpdump :- designed for capturing & displaying packets

ⓐ [root@server ~] # dnf install -y tcpdump

ⓑ [root@ serva ~] # tcpdump -D

ⓒ [root@saver ~] # tcpdump -i eth0 -c 10
capture traffic to & from the host

ⓓ [root@server ~] # tcpdump -i eth0 src host 8.8.8.8
For traffic coming from 8.8.8.

(e) [root@server~]#tcpdump -i eth0 dst host 8.8.8.8
For outbound traffic going to 8.8.8.8

(f) [root@server ~] # tcpdump -i eth0 net 10.1.0.0/24
capture traffic to and from a network

(g) [root@server ~] # tcpdump -i eth0 port 53
capture traffic to and from port members

(5) Ping :-

Ping is a network utility used to check connectivity between two devices on a network. It sends ICMP Echo request messages to a destination host and waits for Echo Reply messages

eg :- ping google.com

Ping 10 times :-

Ping -c 10 google.com

Reply Received : connection is working properly

Destination Host Unreachable : No route to the destination host

Request Timed out : No response received from the host

unknown Host : Hostname is incorrect or cannot be resolved

A good ping result should have low latency and 0.1% packet loss.

Result :- Various network commands used in LINUX & windows are executed successfully

27/8/26

23/7/24

PRACTICAL-2

STUDY OF DIFFERENT TYPES OF NETWORK CABLES

Different type of cables used in networking are :-

1) Unshielded Twisted pair (UTP) cable
2) Shielded twisted pair (STP) cable
3) Coaxial cable
4) Fibre optic cable

Cable Type	Category	Maximum Data transmission	Advantages / Disadvantages	Application/Use
UTP	Category 3	10 bps	Advantages: → Cheaper in cost → Easy to install as they have smaller overall diameter	→ 10 Base T- Ethernet → Fast ethernet, Gigabit ethernet
	Category 5	upto 100 Mbps		
	Category 5e	1Gbps	Disadvantages: → More prone to (EMI) electromagnetic interference & noise	
STP	Category 6, 6a	10Gbps	Advantages:- → Shielded → Faster than UTP → less susceptible to noise & interference	Gigabit ethernet, 10G ethernet (55m) Widely used in data centres
SSTP	Category 7	10Gbps	Disadvantages:- → Expensive → Greater installation effort	Gigabit ethernet, 10G ethernet (100m)

Cable type	Category	Maximum Data transmission	Advantages/ Disadvantages	Application/ use
Co-axial cable	RG-6 RG-59 RG-11	10-100 Mbps	Advantages:- → High bandwidth → Immune to interference → low loss band - width → versatile	Speed of signal is 500m. Television Network High Speed Internet connections
			Disadvantages:- → limited distance → cost → size is bulky	
Fibre optics cable	Single mode Multi Mode	100 Gbps	Advantages:- → High speed → High bandwidth → High security → long distance Disadvantages:- → Expensive → Requires skilled installers.	Maximum distance of fibre optics cable is around 100 meters.

Result:- Thus the different types of network cables are studied successfully.

4/12/24

3/8/21

PRACTICAL - 3

CISCO PACKET TRACER

Aim :- TO study cisco packet tracer tool installation and user interface overview

Introduction :-

A Simulator, as the name suggests, simulate network devices and its environment packet tracer is an existing network design, simulation and modelling tool

→ It helps you to practice your network configuration and trouble shooting skills via computer or an android or iOS based mobile services

→ Protocols in packet tracer are coded to klook and behave in the same way as they would on real hardware

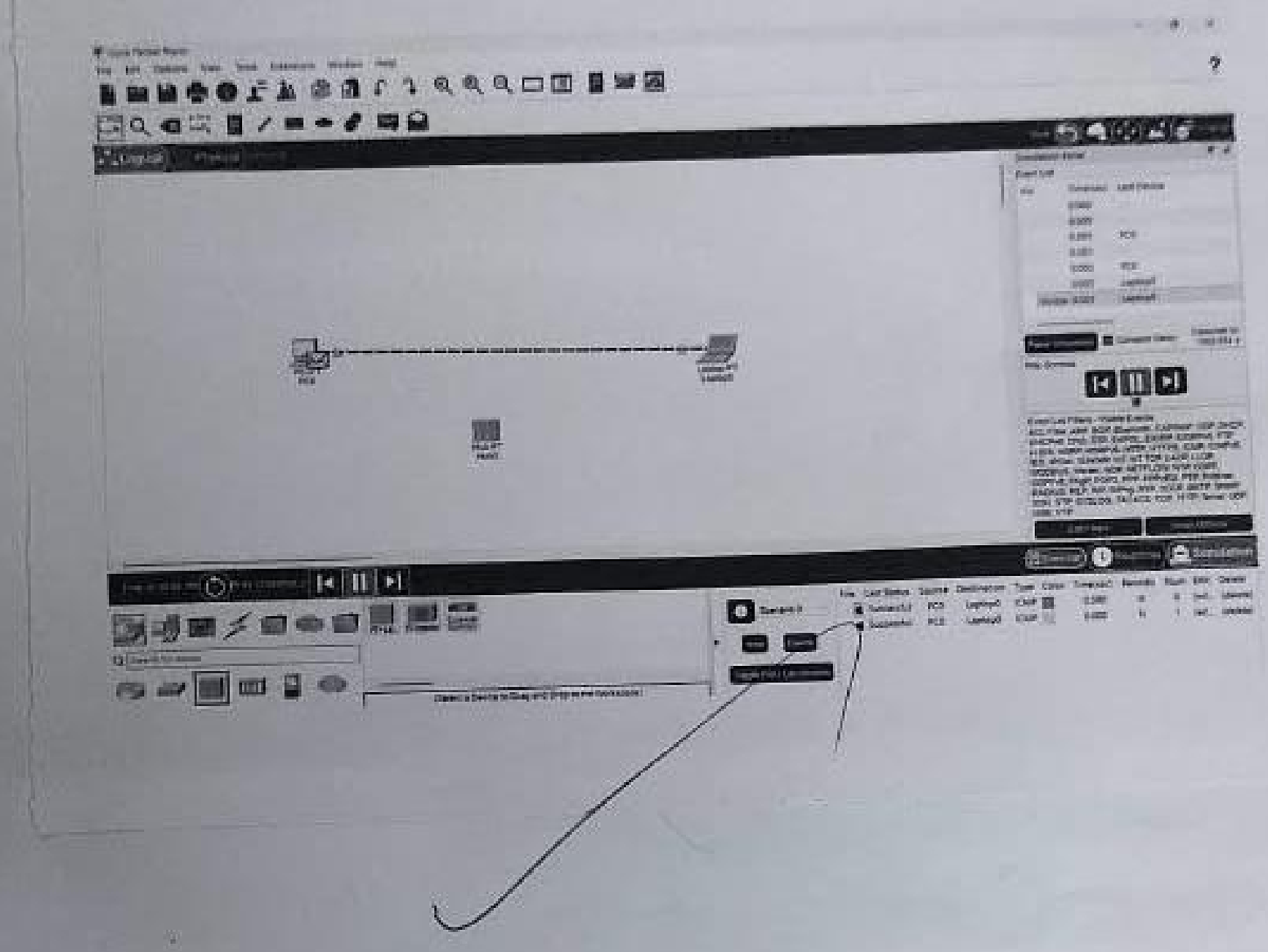
Analyze the behaviour of network devices using cisco packet tracer simulator

i) Peer to peer
ii) Switch
iii) Hub

① Peer to Peer :

A peer to peer network is a network in which computers communicate directly with each other without requiring a dedicated central server

Eg:- TWO PC's connected directly using a cable and communicating with each other

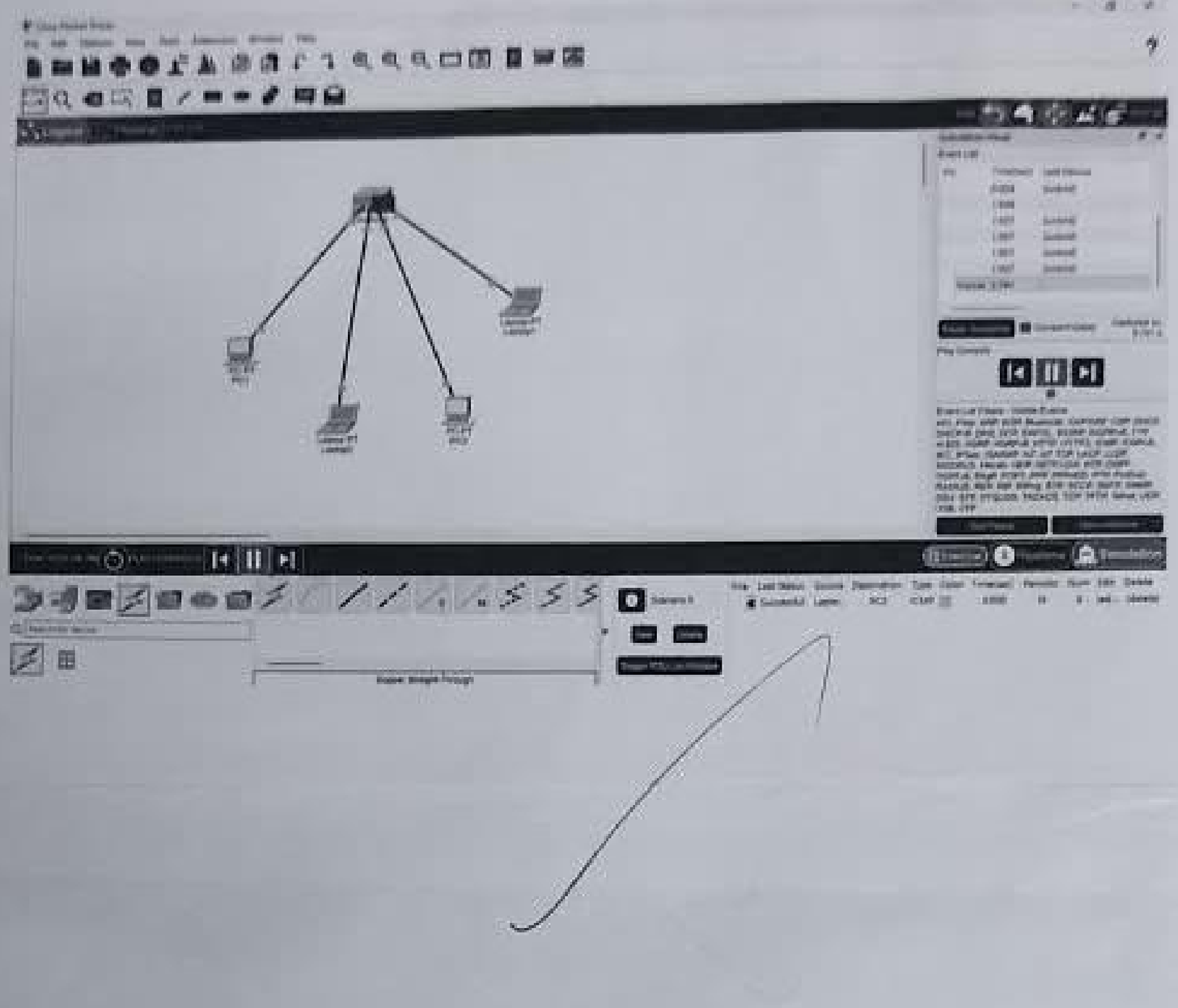


(ii) Switch :-

A switch is a networking device used to connect multiple devices in a network and forward data only to the intended destination device

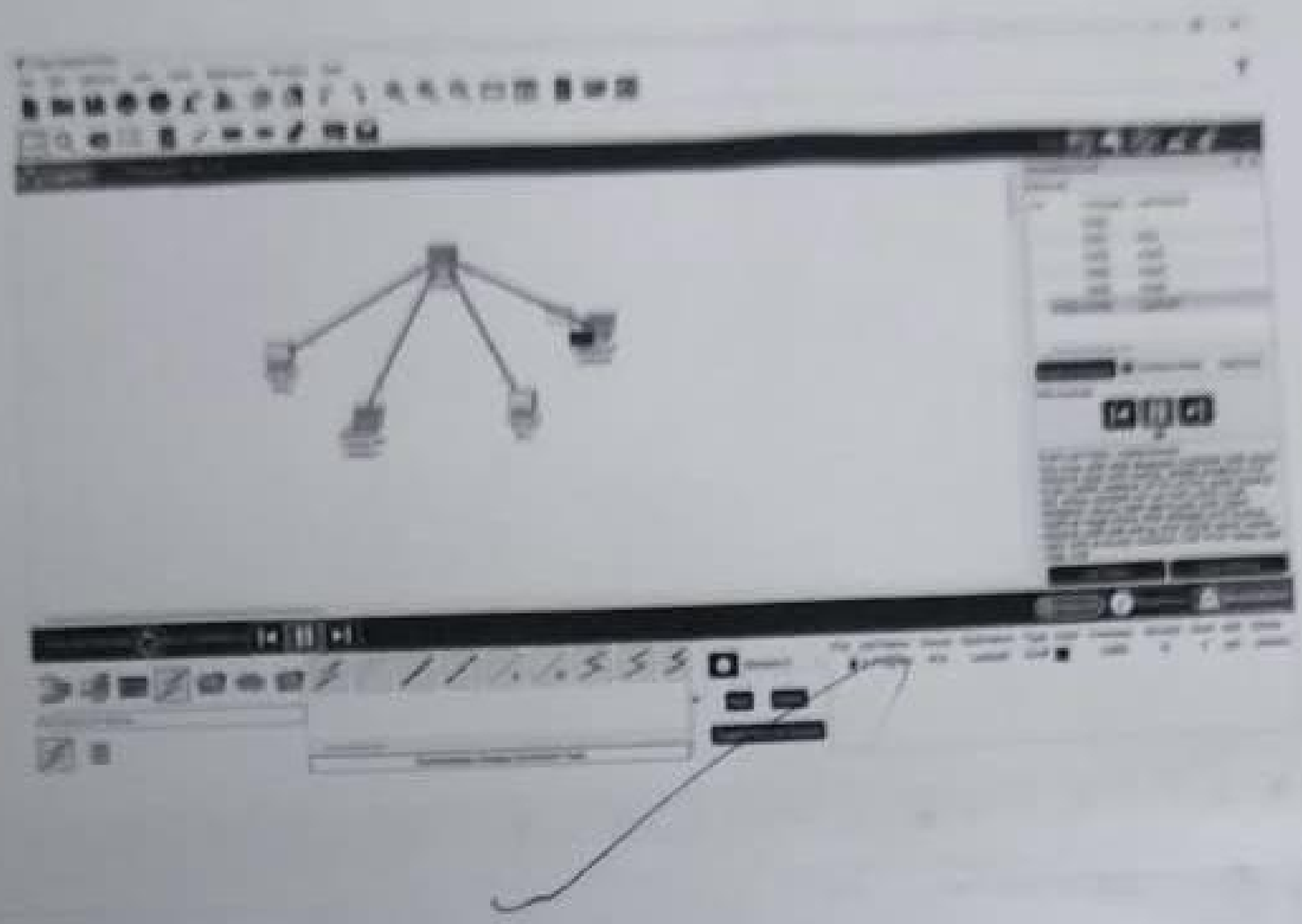
Eg: PC1 → Switch → PC3

Switch knows where PC3 is and forwards the frame only towards PC3. Switch only works at the Data link layer (layer 2) of the OSI model.



(iii) Hub :- A Hub is a basic networking device used to connect multiple computers in a network. It receive data from one device & broadcasts it to all connected devices. Hub works mainly at physical layer (layer 1) of the OSI model

Eg :- If PC1 sends data to PC3 through a hub the hub sends that data to PC2, PC3, PC4 etc



Result : Thus the cisco packet tool is installed successfully & behaviour of network devices using cisco is studied

10/8/24

PRACTICAL - 4

SETUP AND CONFIGURE A LAN USING SWITCH AND ETHERNET CABLES

What is LAN :-

LAN stands for local Area network which is a group of computer and connected devices linked together in a small physical locations like home, school or office building

How to set up LAN :-

Step ① connect the hardware

↳ Place your router in a central location near a power outlet

↳ connect your modem to the router's WAN (or) internet port using an ethernet cable

↳ Plug the router into power and wait for it to turn on

Step ② connect your devices

Wired setup: Plug one end of an ethernet cable into your computer's networks port and the other end into one of the numbered LAN ports on the router

Step ③ configure the router

↳ Type your IP address (192.168.1.1) (or) (192.168.0.1) into the address bar and Press enter

↳ log in using the default username and password printed on the router

Step ④ - check the connectivity between switch and other machine by using ping command in the command prompt of the device

Step ⑤ - Try to access the shared folder from others computers of the network

Result :- Wireless LAN using switch and ethernet cables are executed successfully in cisco Packet tracer.

10/8/24

12/8/24

EXPERIMENT - 5

WIRESHARK

Aim: Experiments on packet capture tool : Wireshark

TO filter, capture, view, packets in wireshark tool capture 100 packets from the ethernet IEEE 802.3 LAN interface and save it

Procedure

↳ Select local area connection in Wireshark
↳ TO capture option
↳ Select stop capture automatically after 100 Packets
↳ Then click start capture
↳ Search TCP packets in search bar
↳ TO see flow graph click statistics flow graph
↳ Save the packets

① create a filter to display only TCP / UDP packets inspects the packets and provide the flow graph

↳ Select LAN connection in Wireshark
↳ TO capture option
↳ Select stop capture automatically after 100 packets
↳ Then click start capture
↳ Save the packets

② create a filter to display only ARP packets and inspect the packets

Procedure

↳ Go to capture option

↳ select stop capture automatically after 100 packets

↳ start & search ARP packets in search bar

↳ save the packets

③ create a filter to display only HTTP packets and inspect the packets procedure

↳ Select local area connection in wireshark

↳ Go to capture option

↳ Select stop capture automatically after 100 packets

↳ Then click start

↳ search HTTP packets in search bar

④ create a filter to display only DNS packets and provide the flow graph

Procedure

↳ Go to capture option

↳ Select stop capture automatically after 100 packets.

↳ Then click start capture

↳ search DNS packets in search bar

↳ To see the flow graph click statistics flow graph

⑤ Create a filter to display only DHCP packets and inspect the packets

↳ Select local area connection in Wireshark

↳ Go to capture option

↳ Select stop capture automatically after 100 packets

↳ click start

↳ search DHCP packets in search bar

↳ Save the packets

Result: Wireshark on packet capture tool experiment are learned and executed successfully.

18/8/25

11/9/24

Exp: 6 Error Detection & Correction using Hamming Code

Aim:-

Write a program to implement error detection and correction using hamming code concept Make a test run to input data stream and verify error correction features.

Sender

↳ Input to sender file should be a text of any length program should be converted to text to binary

↳ Apply hamming code concept on the binary data and add redundant bits to it

↳ Save this output in a file called channel

Code:

```
data = input ("Enter 4 bits: ")
d1, d2, d3, d4 = map (int, data)
P1 = d1 ^ d2 ^ d4
P2 = d1 ^ d3 ^ d4
P4 = d2 ^ d3 ^ d4
code = [P1, P2, d1, P4, d2, d3, d4]
Print ("Sender hamming code:", "".join
                (map(str, code)))
```

Receiver side:

↳ Receive the hamming code from the sender

↳ Enter the received code
↳ calculate the parity check value
↳ find error position using parity values
↳ correct the error by changing the bit
↳ Display the corrected hamming code

Code

```
r = list (map(int, input ("Enter received 7 bits:")))
r1 = r[0] ^ r[2] ^ r[4] ^ r[6]
r2 = r[1] ^ r[2] ^ r[5] ^ r[6]
r3 = r[3] ^ r[4] ^ r[5] ^ r[6]

    error = r1 *1 + r2* 2 + r4* 4
if error == 0;
    Print ("No Error")
else :
    Print ("Error at pontion: ", error)
    r[error-1] ^= 1
Print ("corrected code : ", "".join (map (str, received
```

Input:

Enter 4 data bits : 1011
Sender Hamming code : 0110011
Received 7 bits : 0010011

output :

Error at position : 2
corrected code : 0110011

Sample Input &:-

Enter 4 data bits : 0010

Sender Hamming code : 0101010

Enter received 7 bits : 0101010

Output:-

No Error_

Result :- Error detection & error correction using hamming code was executed successfully.

1 22/9/25

Experiment : 7

NMAP to Discover live Hosts using NMAP Scans on the TryHackme platform

Aim :-

TO Discover live hosts using Nmap scans (ARP, ICMP, TCP/UDP) on the TryHackme platform

Room Link: https:// tryhackme.com/ room/nmapol

Procedure :

Task ① Introduction

↳ ARP Scan : This scan uses ARP requests to discover live hosts

↳ ICMP scan : This scan uses ICMP requests to identify live hosts

↳ TCP/UDP Ping Scan : This scan sends packets to TCP ports and UDP ports to determine live hosts

Task 2 : Subnetworks

Network Segment - It is a collection of computers linked together through a common medium

Subnetworks - It is usually the equivalent of one or more networks segments linked together and configured to utilize the same router in an IP networks

* Subnets with /16
* Subnets with /24

Task 3 : Enumerating Targets

We need to identify the targets we want to scan before we conduct any action

① List

< MACHINE - IP > < scanme nmap sog > < example 10 >

② Range

10 11 12 15 20

③ Subnet

< machine - IP 7/30

→ Scan 3 , 6, 4 IP address

Task 4 :- Discovering live hosts

① ARP , from link layer - commonly used for Man - In - Middle - Attack (ARP Spoofing and ARP poisoning)

② ICMP from network layer - commonly used in DDOS attacks

③ TCP from transport layer - SYN flooding, TCP Reset, TCP session hijacking attacks

④ UDP from transport layer - also ~~not~~ used in DDOS attacks

Task 5 : Nmap Host Discovery using ARP

↳ When a privileged user tries to scan LAN, Nmap uses ARP requests

↳ TO discover internet hosts without port scanning - live systems use nmap -sn targets

↳ nmap -PR -SN < TARGETS >

↳ nmap -PR -SN < MACHINE . &P>/24

↳ nmap -PR -SN < ATTACKBOX . &P/24

Task 6: Nmap host discovery using ICMP

① Add -PE option to utilize ICMP echo requests to discover live hosts

② nmap -PE -sn TARGET to check host is live

③ -PP option instructs nmap to use ICMP timestamp requests

④ nmap -PP -SN TARGET

-PM option can enable this scan to ICMP address mask requests

Task 7:- Nmap host discovery using TCP and UDP

↳ Nmap to use TCP SYN Ping use -PS followed by the port number, range, list or a combination of this option

-PS21, -PS25 targets ports 21, 22, 23, 24, 25

↳ nmap -PS, -SN TARGET Check the TCP port is open

↳ TCP Ack Ping - running Nmap as privileged user

↳ Nmap -PA -SN TARGET

↳ UDP Ping - sending UDP packet to an open port is unlikely to result in a response

↳ If we transmit a UDP packet to a closed UDP port we should expect to receive an ICMP port unreachable packet

nmap -PU -SN TARGET

Task 8 : Using Reverse -DNS LOOK up

↳ -R option to query the DNS sever even for offline hosts

Task 9 : Summary

ARP Scan, ICMP Echo Scan, ICMP timestamp Scan, ICMP address mask scan, TCP SYN Ping Scan, TCP Ack Ping Scan, UDP Ping Scan.

Result :

Nmap to discover live host using tryhackme platform was successfully executed.

22/9/22

1 28/9/26

EXPERIMENT : 8

End to End communication at transport layer

Aim: Implement echo client server using TCP / UDP Packets

Procedure:

↳ Create Socket using python Socket Programming

↳ Specify the IP address and port number

↳ Bind the server socket to IP address & port

↳ Make server listen for client Communication

↳ connect the client to server

↳ Send & receive message between client and Server

↳ Display the received merage

↳ Close the Socket connection

code

TCP Server

```
import socket
Server = socket.socket(socket.AF_INET, socket.
                                    SOCK_STREAM)
server.bind("127.0.0.1", 5000)
Server.listen(1)
Print("Server is waiting for connection..")
```

```
conn addr = Server.accept()
Print("client connected")

While true:
    data = conn.recv(1024).decode()
    if not data:
        breaks
    Print("Client:", data)
    Conn.send(data.encode())
    Conn.close()
    Server.close()
```

TCP - Receiver (client)

```
import Socket
client = Socket.Socket(Socket.AF_INET,
                       Socket.SOCK_STREAM)
Client.connect(("127.0.0.1", 5000))
message = input("Enter message: ")
Client.send(message.encode())
reply = Client.recv(1024).decode()
Print("Server:", reply)
Client.close()
```

output

Terminal 1 server

Python tcp_server.py

Server is waiting for connection...

Client connected

client : Hello

Terminal 2 client

Python tcp_client.py

~~Python tcp_client~~

Enter message : Hello

Seever : Hello

TCP echo is working

Result :-

The TCP/UDP packets using echo client server socket program is executed successfully

28/9/24

28/9/21

EXPERIMENT - 9

SLINDING WINDOW PROTOCOL

Aim :-

Write a program to implement flow control at data link layer using sliding window protocol. Simulate the flows from one node to another

Code

```
import time
window = int (input (" Enter window size : "))
message = input (" Enter message : ")
start = 0
while Start < len ( message )
Print (" SENDER ")
end = start + window
if end > len ( message):
    end = len (message)
# sender buffer
  Sender - buffer = [ ]
  for i in range ( start, end) :
      frame = (i, message [i ])
      Sender - buffer. append ( frame )
  Print (" Frame NO : ", i, " DATA : ", message [i])
Print (" In Sender Buffer : ")
Print ( Sender. Buffer )
```

```
# Delay
Print ( "\n waiting for receiver ")
time.Sleep (6)
# Receiver
Print (" RECEIVER ")
expected = start
error False
for frame in sendes_buffer:
frame_no = frame [0]
data = frame [1]
Print (" Recived frame : ", frame_no, "DATA:", data
if frame_no == expected :
    expected = expected +1
else :
    error = True
    break
# Receiver buffer
if error:
    receiver_buffer = ("NACK", expected)
    Print (" In Receiver Buffer: ")
    Print( receiver_buffer)
else:
    ack = expected - 1
    receiver_buffer = ("ACK", ack)
    Print(" In Receiver buffer: ")
    Print(" receiver_buffer ")
```

```
# Sender reads ACK/NACK
    Print("IN SENDER CHECKING RESPONSE")
    response = receiver_buffer[0]
    number = receiver_buffer[1]
if response == "ACK":
    Print("ACK received:", number)
    start = number + 1
else:
    Print("NACK received:", number)
    start = number
Print("TRANMISSION COMPLETED")
```

OUTPUT:

Enter window size: 3

Enter message: Hello

SENDER

Frame NO: 0 DATA: H

Frame NO: 1 DATA: e

Frame NO: 2 DATA: l

Sender Buffer:

[(0,'H'), (1,'e'), (2,'l')]

Waiting for receiver...

RECEIVER

Received Frame: 0 DATA: H

Received Frame: 1 DATA: e

Received Frame: 2 DATA: l

Receiver Buffer:

('ACK', 2)

SENDER CHECKING RESPONSE

ACK Received: 2

SENDER

Frame no: 3 DATA: l

Frame no: 4 DATA: O

Sender Buffer:

[(3, 'l'), (4, 'O')]

Waiting for receiver..

RECEIVER

Received frame: 3 DATA: l

Received frame: 4 DATA: O

Received Buffer: ('ACK', 4)

SENDER CHECKING RESPONSE

ACK Received: 4

TRANSMISSION COMPLETED

28/9/22

Result :- Sliding Window protocols are learned and executed successfully

29/10/24

EXPERIMENT : 10

Implementation of subnetting in CISCO PACKET TRACER Simulator

Aim:- TO implement subnetting using CISCO packet tracer and verifying communication between the subnets

Subnetting :-

It is the process of dividing large network into smaller logical networks called subnets. It helps in efficient utilization of IP address, reduces unnecessary network traffic & improve network management.

Algorithm :-

↳ start cisco Packet tracer

↳ Place one router, two switches & four PCs in the workspace

↳ connect 2PC to switch & both switches to the router

↳ Assign static IP address & subnet masks to all PCs

↳ configure IP address on the router interfaces connected to the two subnets

↳ Configure default gateway for each Pc

↳ Check connectivity between Pcs with same subnet using ping command

↳ Test communication between Pcs belonging to different subnets

↳ Disconnect (or) disable the router interfaces & repeat communication test

↳ observe & record result

Procedure

Step ① Design Network Topology

↳ connect PC0 & PC1 to Switch 0

↳ Connect PC2 & PC3 to Switch 1

Step ② Assign IP address

Device	IP address	Subnetmask	Default Gateway
PC0	192.168.1.2	255.255.255.0	192.168.1.1
PC1	192.168.1.3	255.255.255.0	192.168.1.1
PC_2	192.168.2.2	255.255.255.0	192.168.2.1
PC_3	192.168.2.3	255.255.255.0	192.168.2.1

Step ③ Configure the Router

↳ open CLI

↳ Terminal

enable
configuration terminal

interface gigabitEthernet 0/0
ip address 192.168.1.1 255.255.255.0
no shutdown
exit

interface gigabitEthernet 0/1
ip address 192.168.2.1 255.255.255.0
no shutdown
exit
End

Step ④ :- Test communication between same subnet

PC0, Ping 192.168.1.3

Check PC1 responds

Step ⑤ :- Test communication between different subnets

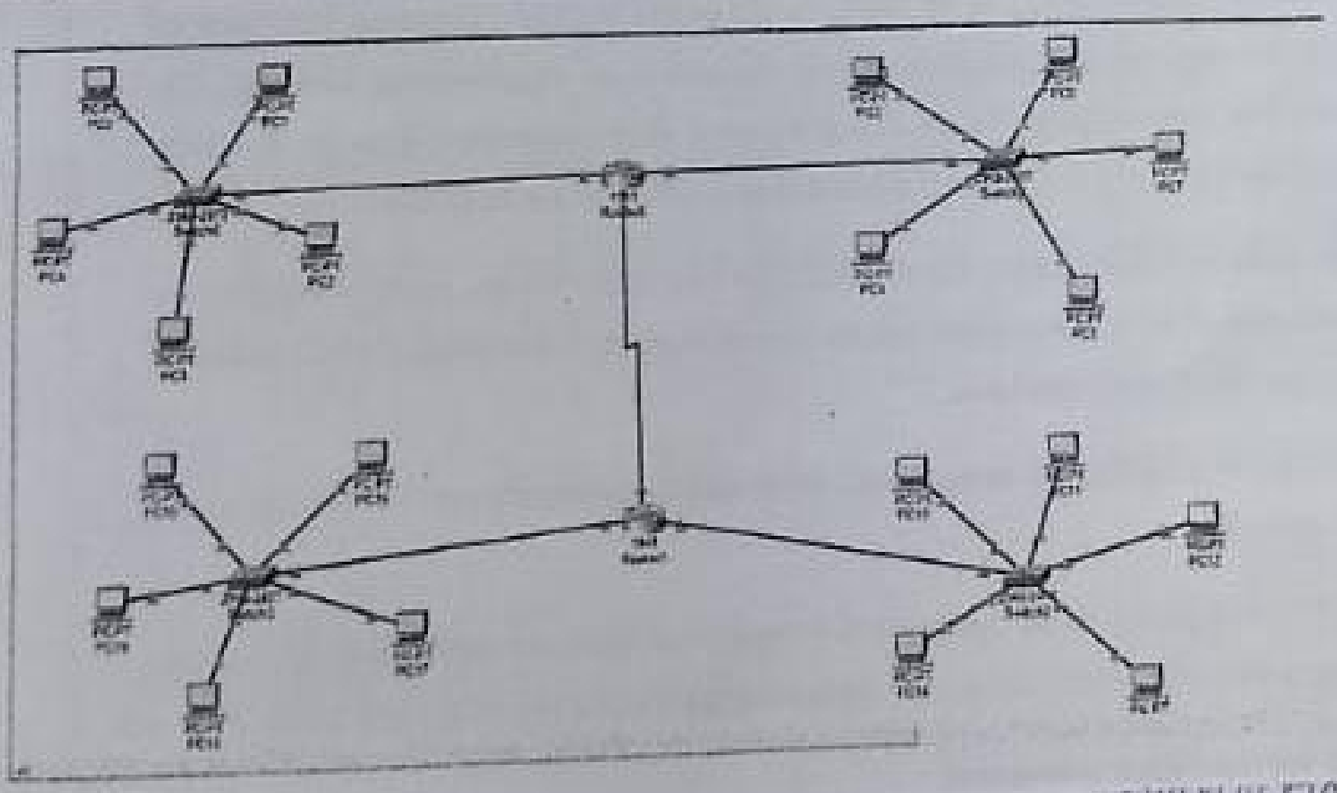
PC0, ping 192.168.2.2

Check whether PC2 responds

Step ⑥ : Test communication without Router

↳ Disconnect router links

↳ [illegible] PC0 again



sult

when router is
ed & connected

& when the router
connection between subnets
is disabled

Result : Thus the subnetting was implemented successfully using cisco packet tracer

5/10/22

EXPERIMENT - 11

Internetworking with Routers in CISCO PACKET TRACER Simulator

10 (a) Internetworking using Router

Aim: To design and configure simple internetwork using router in cisco Packet tracer

Algorithm:

- Place router & two PCs in packet tracer
- Connect PCs to router using straight through cables
- Assign IP address & subnet masks to PC & router interface
- Configure router using CLI & enable them
- Use ping to test connectivity

Procedure:-

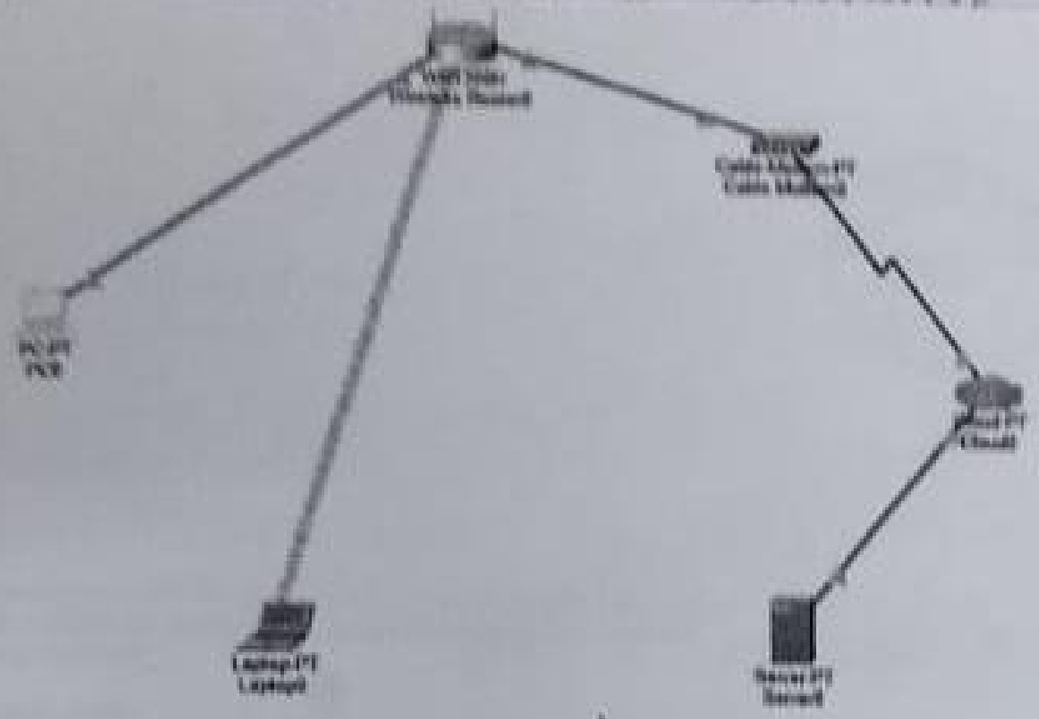
- Open packet tracer & create network topology
- Configure router interfaces with IP addresses
- Assign IP address to the PC in different Network
- Enable router interface using no shutdown
- Test communication between PCs

Result: The internetwork was configured successfully. & connectivity between PCs was tested using router

10 (b) Wireless Router, DHCP server & Internet cloud

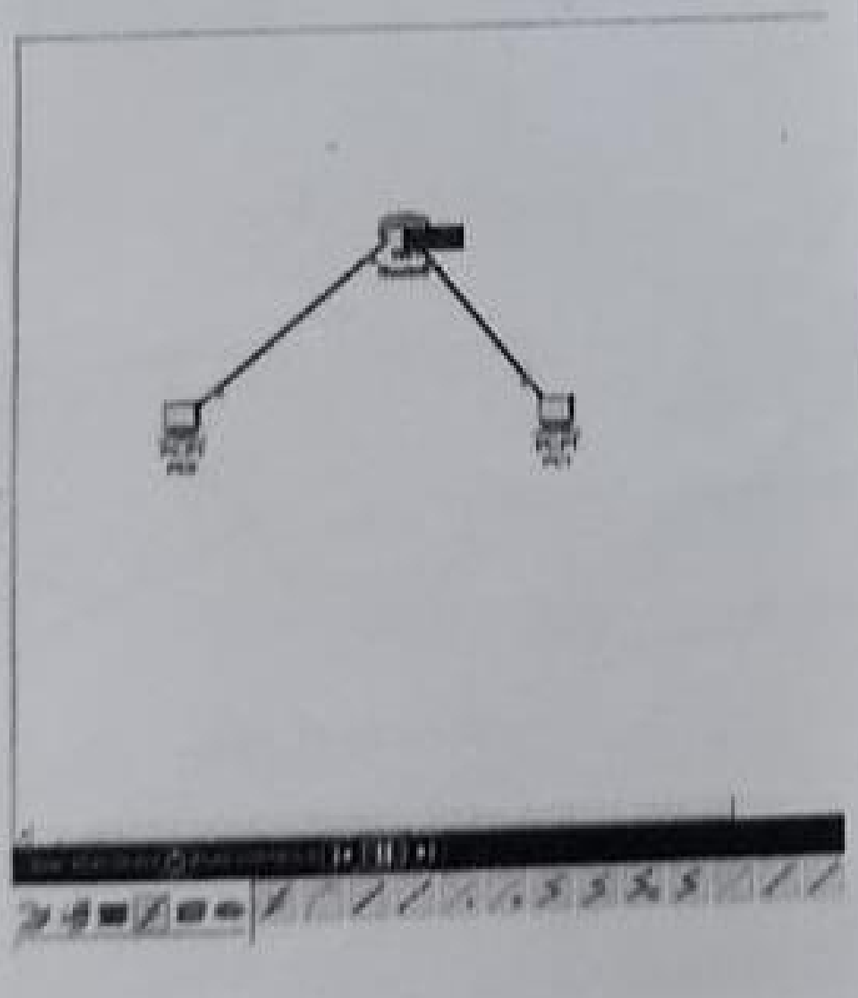
Aim :-

...n internetwork using
& Internet cloud



PCs & Internet cloud
work & DHCP service
s settings
addres automatically
og command

Procedure



opology in packet tracer
configure Wireless router
with IP & DNS settings
eless network
s
using ping
ng ping cisco.com

Result :- The Wireless Internetwork was configured, & DHCP and network connectivity were tested

10 (c) Internetwork using switch, Router and Ethernet cables

Aim: To design and configure an internetwork using switches, router & Ethernet cables

Algorithm:

- ↳ Place the router, switch & PCs in packet tracer
- ↳ connect PCs to switch & switch to router
- ↳ Assign IP address, subnet masks & default gateways
- ↳ Configure & enable the router interfaces
- ↳ Test connectivity using PING

Procedure:

- ↳ create network topology
- ↳ connect devices using ethernet cables
- ↳ Configure IP address
- ↳ Enable router interfaces
- ↳ Verify communication

Result: The internetwork was configured successfully, & connectivity between devices was verified

13/10/26

EXPERIMENT - 12

Routing at Network layer

12(a) Simulate static Routing Protocol configuration using Cisco Packet tracer

Aim : To simulate static routing configuration using Cisco packet tracer

Algorithm :-

↳ place routers & Pcs in Packet tracer
↳ connect the devices & assign IP address
↳ configure the router interfaces & enable them
↳ Add static routes using IP route command
↳ Test connectivity using Ping and tracert Commands
↳ Use tracer to verify the routing protocol
↳ Thus, a static routing is successfully configured and verified.

Result : Thus, the static routing was configured successfully & connectivity between different networks was verified.

12(b) RIP using cisco packet tracer

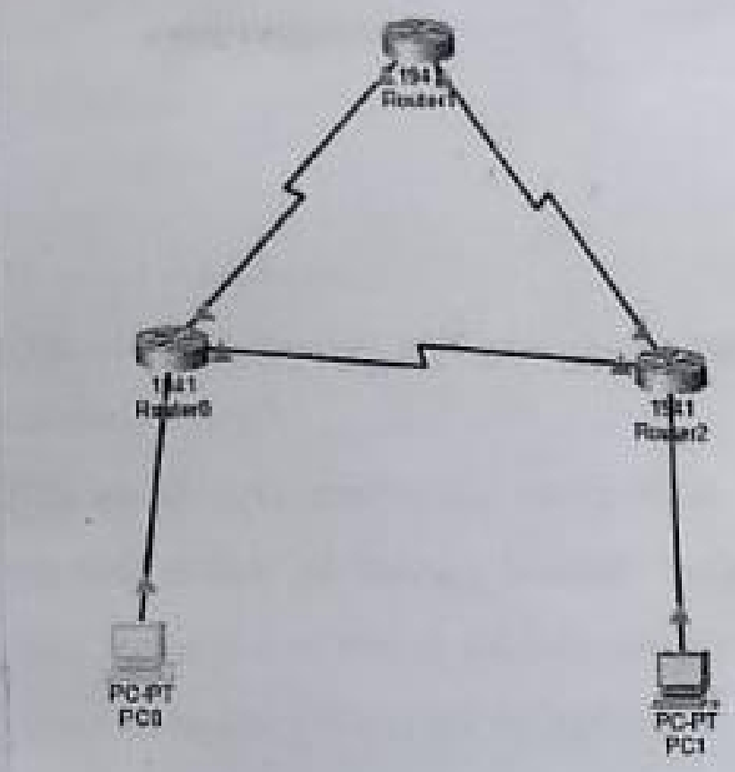
Aim: To simulate the routing information protocol (RIP) using cisco packet tracer

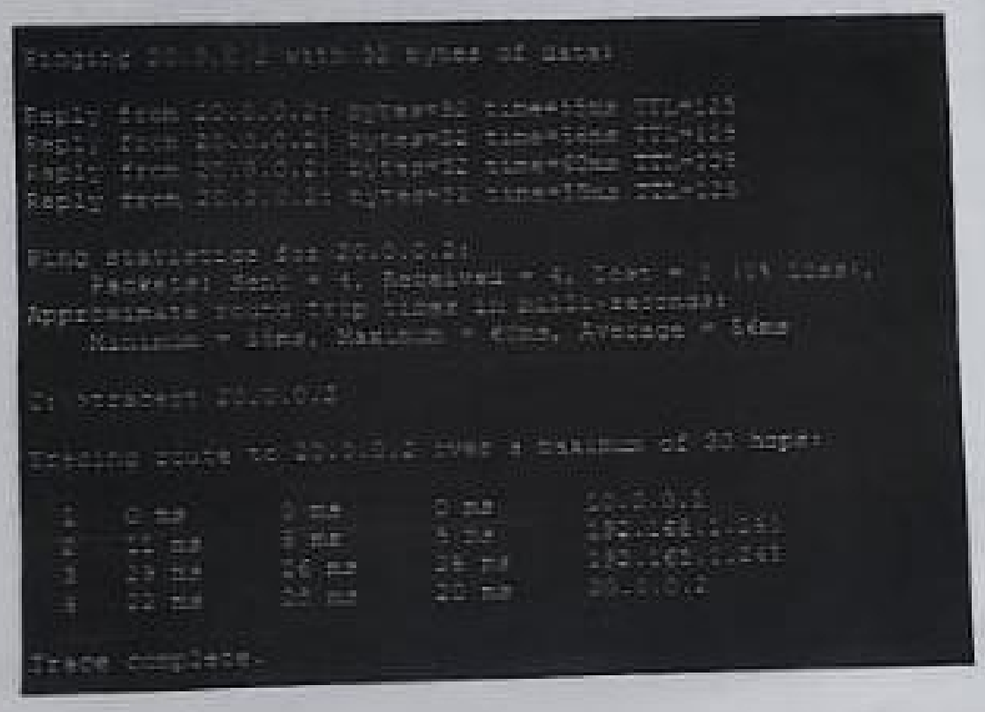
Algorithm:

↳ Place routers and PCs in Packet tracer
↳ connect the devices and assign IP addres
↳ configure and enable router interfaces
↳ Enable RIP & advertise the connected networks
↳ Test connectivity between PCs using Ping

Procedure:

↳ Create a network topology with multiple routers & PCs
↳ Configure IP address & Subnet masks
↳ Enable router interfaces using 'no shutdown'
↳ Enter RIP configuration mode using 'router rip' and advertise the networks using the network command
↳ verify the routing table using 'show ip route' and test connectivity





```
Router>enable
Router#show ip route static
S    30.0.0.0/8 [20/0] via 40.0.0.2
S    50.0.0.0/8 [10/0] via 40.0.0.2
S    60.0.0.0/8 [1/0] via 40.0.0.2

Router#
%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/0, changed state to up

%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/0, changed state to down

%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/0, changed state to up

Router#show ip route static
S    30.0.0.0/8 [10/0] via 20.0.0.2
S    50.0.0.0/8 [10/0] via 40.0.0.2
S    60.0.0.0/8 [1/0] via 40.0.0.2
```

Result: RIP was configured successfully, and communication between different networks was verified using dynamic routing

24/04/26

13/10/24

EXPERIMENT - 13

Implement your own Ping Program

Aim :-

To write a python program to implement the ping utility and check network connectivity between two hosts

Algorithm :-

↳ Get destination IP address from user
↳ Send an ICMP echo request using the System Ping command
↳ Receive the response from the destination
↳ Display whether host is reachable
↳ Stop the program

Code

```
import subprocess
import platform

ip = input("Enter destination IP address: ")
if platform.system().lower() == "windows":
    command = ["ping", "-n", "4", ip]
else:
    command = ["ping", "-c", "4", ip]
result = subprocess.run(command, capture_output=True,
                        text=True)
print(result.stdout)
```

```
if result.returncode == 0:
        Print("Host is reachable")
else:
        Print("Host is unreachable")
```

Output :-

I/P :-

Enter destination IP address:

8.8.8.8

O/P :-

Reply from 8.8.8.8

Host is reachable

Result : Thus, the python program to check network connectivity using ping utility was executed successfully

31/07/20

12/10/24

EXPERIMENT - 14

Packet Sniffing using Raw Sockets

Aim : To write python program using raw sockets to capture network packets and display their source IP, destination IP and Protocol

Algorithm :-

- ↳ create raw sockets
- ↳ capture incoming IP packets
- ↳ Extract IP header from each packet
- ↳ Identify source IP address & destination IP address
- ↳ Identify protocol used
- ↳ Display packet information

Code

```
import socket
import struct
try:
    Sniffer = socket.socket(
        socket.AF_INET,
        socket.SOCK_RAW,
        socket.IPPROTO_IP
    )
except OSError as e:
    Print ("Socket creation failed: ", e)
    Print ("Run with administrator/root privileges.")
    raise SystemExit
Print(" Packet Sniffer started. Press ctrl+c to stop.")
```

```
try:
    while True:
        packet, address = sniffer.recvfrom(65535)
        if len(packet) < 20:
            continue
        version_ihl = packet[0]
        version = version_ihl >> 4
        ihl = (version_ihl & 0xDF) * 4
        if version != 4 or len(packet) < ihl:
            continue
        ttl = packet[8]
        protocol = packet[9]
        src_ip = socket.inet_ntoa(packet[12:16])
        dst_ip = socket.inet_ntoa(packet[16:20])
        protocols = { 1: "ICMP",
                      6: "TCP",
                      17: "UDP"
                    }
        print("\n Packet captured")
        print("Source IP:", src_ip)
        print("Destination IP:", dst_ip)
        print("Protocol:", protocols.get(protocol, str(protocol)))
        print("TTL:", ttl)
        print("Packet size:", len(packet))
except KeyboardInterrupt:
        print("\n Packet sniffing stopped")
finally:
    sniffer.close()
```

Output:

Packet sniffer started.
Press ctrl+c to stop

Packet captured

Source IP : 192.168.1.10
Destination IP : 192.168.1.1
Protocol : TCP
TTL : 64
Packet size : 60

Result: Thus, the python program using raw sockets was implemented to capture network packets executed successfully.